Hochschule Bochum
TECHNIK WIRTSCHAFT GESUNDHEIT

Objektorientierte Programmiertechniken

```
<?>
<? super [Type]>
<? extends [Type]>
<? extends [Type1] & [IType2] & [IType3]>
```

https://www.torsten-horn.de/techdocs/java-generics.htm#Wildcards

Prof. Dr. Ursula Oesing

Vertiefung Java

- Vorspann
- Konventionen
- for each
- Generics
- Funktionale Interfaces / Lambda-Ausdrücke
- Funktionale Programmierung
- Fazit

Vorspann

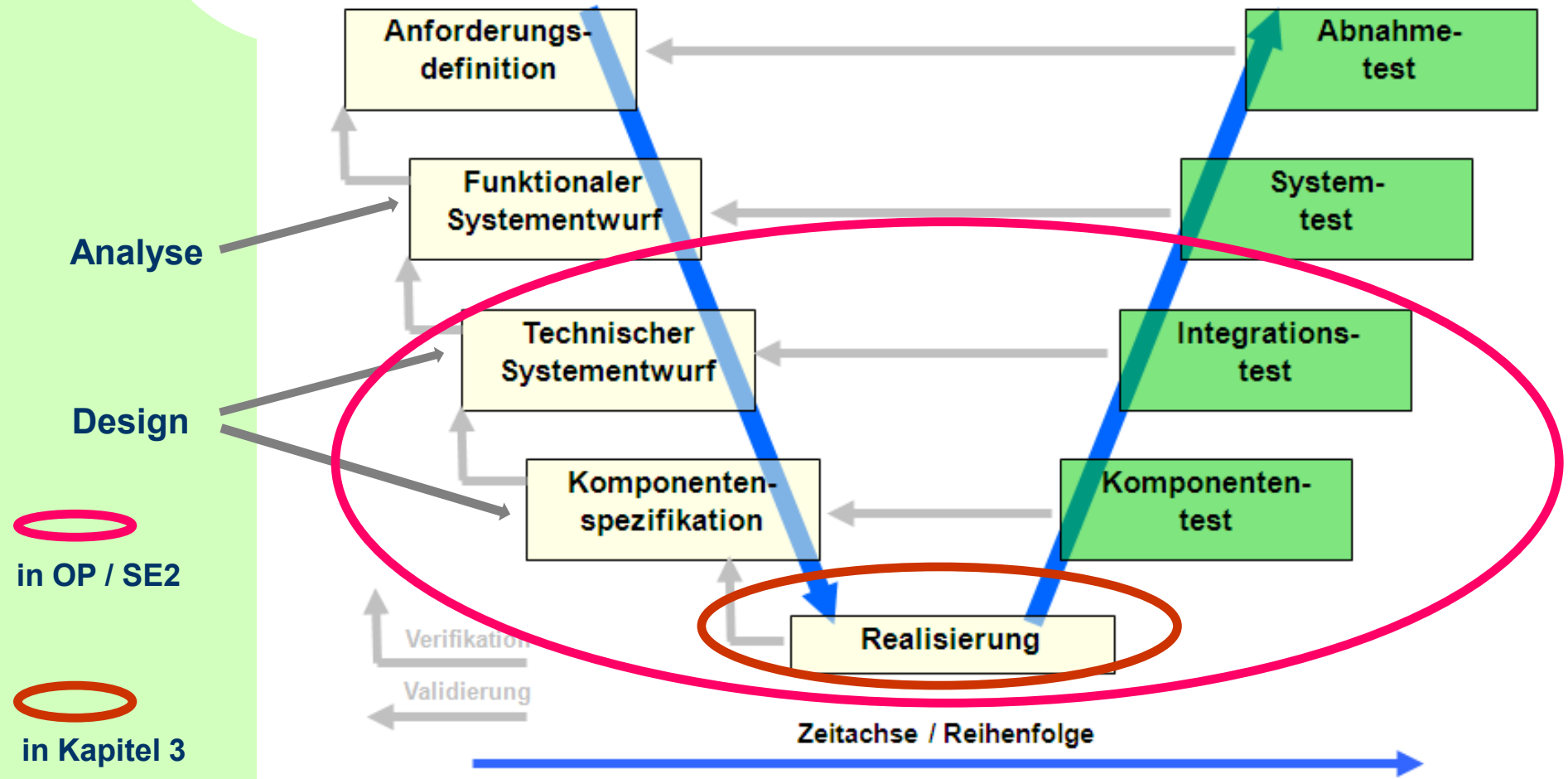
Wieso Vertiefung Java?

- **Konventionen** werden in der Zusammenarbeit im Entwicklungs-team benötigt!
- Weiterentwicklungen in Java haben sich in der Praxis als notwendig erwiesen:
  - umständliche Programmierung von Fachlichkeit, da bisher keine geeigneten Datentypen vorhanden (**Enumeration**, **Generics**)
  - umständliche Programmierung, da bisher keine abkürzenden Techniken vorhanden (**for each**, **Lambda-Ausdrücke**)
- Mit der **Funktionalen Programmierung** ist ein ein neuer fundamentaler Programmierstil eingeführt worden.
  Ihnen soll bekannt sein, dass es diese Weiterentwicklungen gibt!

Vorspann
Konventionen
for each
Generics
Funkt. Interfaces / Lambda-Ausdrücke
Funktionale Programmierung
Fazit

Vorspann

- **Einordnung der Veranstaltung**



Konventionen

Vorspann

Konventionen

for each

Generics

Funkt. Interfaces / Lambda-Ausdrücke

Funktionale Programmierung

Fazit

Java Knigge

- Quellcode soll lesbar sein!
- Konventionen sollen eingehalten werden!

Klassennamen

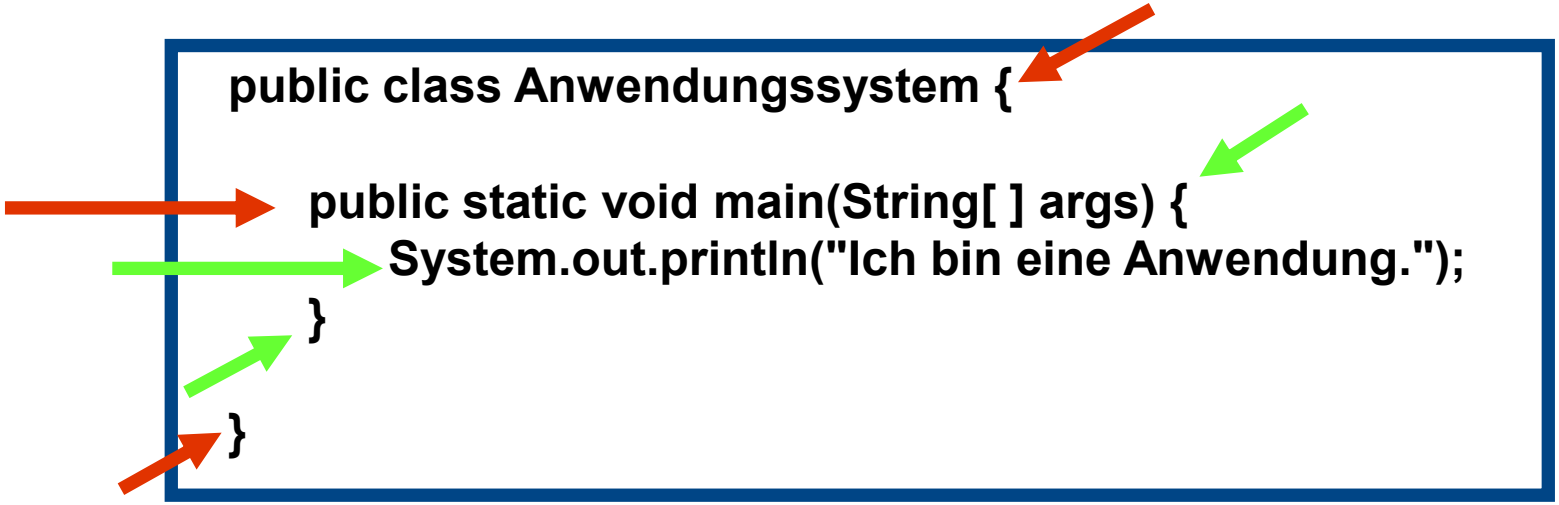
- Sie beginnen mit einem Großbuchstaben und enthalten üblicherweise nur Buchstaben. Vermeiden sie Umlaute und ß. Wählen Sie sprechende Namen!
- Substantiv, meistens im Singular
- Falls der Name aus zwei Worten besteht:

```
public class Anwendungssystem
{
}
```

```
public class SchoeneAnwendung{
}
```

Konventionen

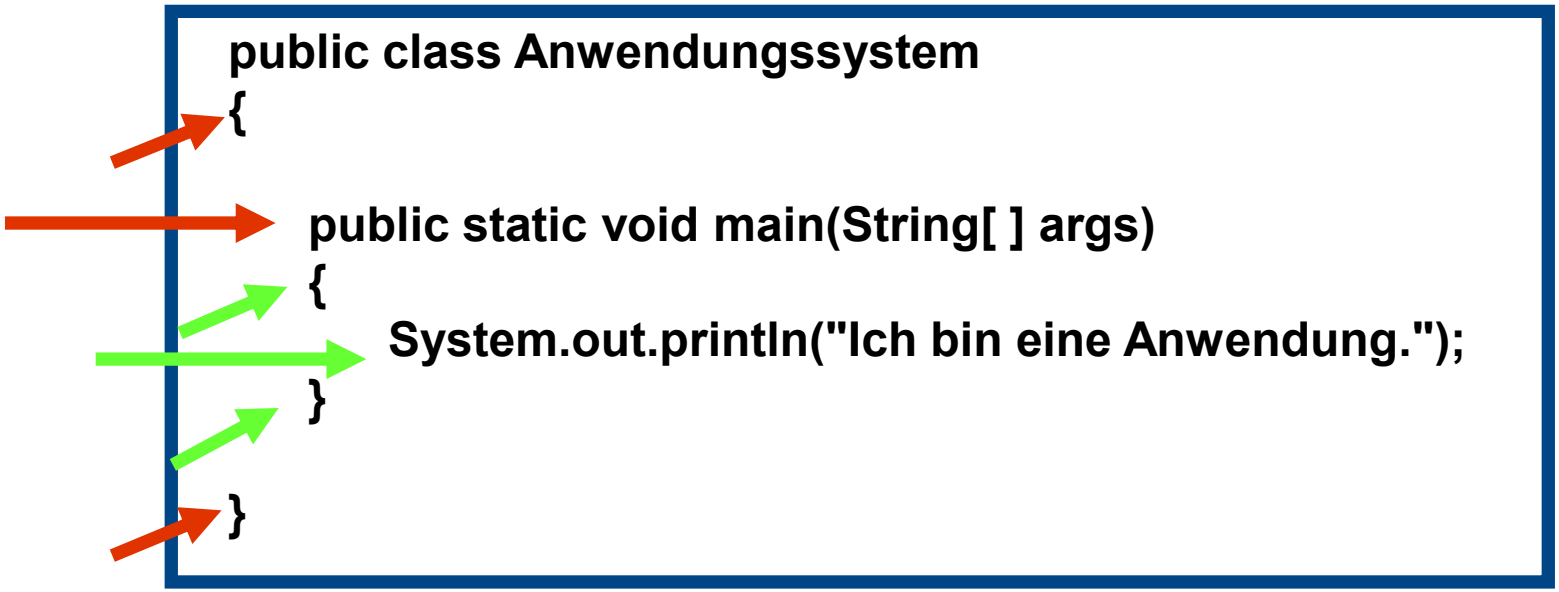
- **Java-Knigge**
- **Klammern und Einrückungen**
  - Bei jeder Klammer zu Beginn eines Abschnitts wird eine Einrückung vorgenommen bis zur Klammer am Ende eines Abschnitts.



Vorspann
Konventionen
for each
Generics
Funkt. Interfaces / Lambda-Ausdrücke
Funktionale Programmierung
Fazit

Konventionen

- **Java-Knigge**
- **Variante: Klammern und Einrückungen**
  - Bei jeder Klammer zu Beginn eines Abschnitts wird eine Einrückung vorgenommen bis zur Klammer am Ende eines Abschnitts.



Vorspann
Konventionen
for each
Generics
Funkt. Interfaces / Lambda-Ausdrücke
Funktionale Programmierung
Fazit

Konventionen

- **Java Knigge**
- **Variablennamen und Attributnamen**
  - Sie beginnen mit einem Kleinbuchstaben und enthalten üblicherweise nur Buchstaben und Ziffern. Vermeiden sie Umlaute und ß. Wählen Sie sprechende Namen!
  - Substantiv, meistens im Singular
  - Falls der Name aus zwei Worten besteht:

```
public class Person {

    private String vorname;
    private String zweiterVorname;
    private String nachname:

}
```

Vorspann
Konventionen
for each
Generics
Funkt. Interfaces / Lambda-Ausdrücke
Funktionale Programmierung
Fazit

Konventionen

- **Java Knigge**
- **Namen von öffentlichen Konstanten**
  - Sie bestehen komplett nur aus Großbuchstaben, ggf. Unterstrich und Ziffern. Vermeiden sie Umlaute und ß. Wählen Sie sprechende Namen!
  - Falls der Name aus zwei Worten besteht, trennt man diese durch einen Unterstrich:

```
public … class Character … {

    public static final char MIN_VALUE = '\u0000';
    public static final byte UNASSIGNED = 0;

}
```

Vorspann
Konventionen
for each
Generics
Funkt. Interfaces / Lambda-Ausdrücke
Funktionale Programmierung
Fazit

Konventionen

- **Java Knigge**
- **Namen von Methoden**
  - Sie beginnen mit einem Kleinbuchstaben und enthalten üblicherweise nur Buchstaben und Ziffern. Vermeiden sie Umlaute und ß. Wählen Sie sprechende Namen!
  - Der Name sollte mindestens aus zwei Worten bestehen, ein Verb im Imperativ (falls in deutsch) (Was macht die Methode?) und ein Substantiv (Womit, macht die Methode etwas oder was ist das Ergebnis?).

```
public class Person {
    ...
    public String getVorname(){
        ...
    }
}
```

Vorspann
Konventionen
for each
Generics
Funkt. Interfaces / Lambda-Ausdrücke
Funktionale Programmierung
Fazit

Konventionen

- **Java Knigge**
- **Wieso Namenskonventionen?**

Vorspann

Konventionen

for each

Generics

Funkt. Interfaces / Lambda-Ausdrücke

Funktionale Programmierung

Fazit

- int geburtsjahr = Integer.parseInt("1970");

for each

- **for each-Schleife**
  - Schleife über eine Menge von Elementen.
  - In Java ist das Schlüsselwort (wie bisher) for.
  - Seit Java Version 1.5.
- **for each und Array**

```
String[ ] nachnamen = {"Brabender", "Blunck", "Oesing"};
for(int i = 0; i < nachnamen.length; i++){
    System.out.println(nachnamen[i]);
}
```

```
String[ ] nachnamen = {"Brabender", "Blunck", "Oesing"};
for(String nachname : nachnamen){
    System.out.println(nachname);
}
```

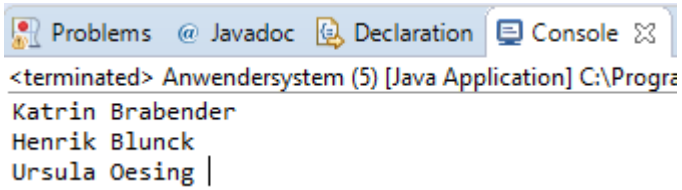
Vorspann
Konventionen
for each
Generics
Funkt. Interfaces / Lambda-Ausdrücke
Funktionale Programmierung
Fazit

for each

Aufgabe

- Gegeben ist ein Beispiel mit einem zweidimensionalen Array.

```
String[ ][ ] namen = {{"Katrin", "Brabender"},
    {"Henrik", "Blunck"}, {"Ursula", "Oesing"}};
for(int i = 0; i < namen.length; i++){
    for(int j = 0; j < namen[i].length; j++){
        System.out.print(namen[i][j] + " ");
    }
    System.out.println("");
}
```



Ändern Sie das vorliegende Beispiel ab. Es sollen for each – Schleifen verwendet werden.

Vorspann

Konventionen

for each

Generics

Funkt. Interfaces / Lambda-Ausdrücke

Funktionale Programmierung

Fazit

for each

Vorspann
Konventionen
for each
Generics
Funkt. Interfaces / Lambda-Ausdrücke
Funktionale Programmierung
Fazit

for each-Schleife

- zusammen mit Container-Klassen (, die das Interface Collection implementieren)

for each und ArrayList

```
ArrayList<String> nachnamen
    = new ArrayList<>();
nachnamen.add("Brabender");
nachnamen.add("Blunck");
nachnamen.add("Oesing");
for(int i = 0; i < nachnamen.size(); i++){
    System.out.println(nachnamen.get(i));
}
```

```
ArrayList<String> nachnamen
    = new ArrayList<>();
nachnamen.add("Brabender");
nachnamen.add("Blunck");
nachnamen.add("Oesing");
for(String nachname : nachnamen){
    System.out.println(nachname);
}
```

Aufgabe

- Erstellen Sie ein Beispiel für for each und eine ArrayList<Integer>, welche Geburtsjahre enthält.

Generics

Vorspann

Konventionen

for each

Generics

Funkt. Interfaces / Lambda-Ausdrücke

Funktionale Programmierung

Fazit

Was ist generische Programmierung?

- Verfahren zur Entwicklung wiederverwendbarer Software. Funktionalitäten (Klassen, Methoden) werden möglichst allgemein entworfen, um sie für unterschiedliche Datentypen verwenden zu können.

  vgl. https://de.wikipedia.org/wiki/Generische_Programmierung

Was sind Generics?

- Sprachmittel, mit denen Klassen und Methoden mit Typparametern parametrisiert werden können, um Typsicherheit trotz generischer Programmierung zu ermöglichen

  vgl. https://www.torsten-horn.de/techdocs/java-generics.htm

Generics

Beispiele

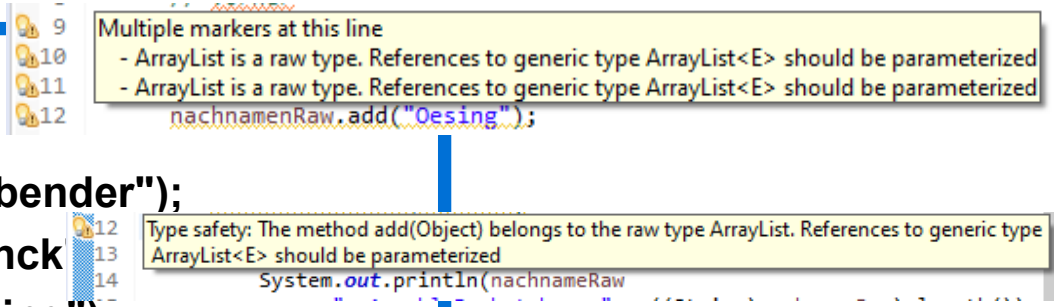
- In der Übung:
  Vector<Observer>
  EventHandler<ActionEvent>
- In der Vorlesung:
  ArrayList<String>
  ArrayList<Integer>
- Die Container-Klassen Vector und ArrayList bieten Funktionalitäten, die für Elemente unterschiedlichen Datentyps zur Verfügung stehen sollen. Es soll aber gewährleistet sein, dass alle Elemente eines Vectors bzw. einer ArrayList denselben Datentyp haben (Typsicherheit).
- Das Interface javafx.event.EventHandler steht für Events unterschiedlichen Datentyps zur Verfügung.

Vorspann
Konventionen
for each
Generics
Funkt. Interfaces / Lambda-Ausdrücke
Funktionale Programmierung
Fazit

Generics

Beispiel ArrayList

```
ArrayList nachnamen
    = new ArrayList();
nachnamen.add("Brabender");
nachnamen.add("Blunck
nachnamen.add("Oesing");
for(Object nachname : nachnamen){
    System.out.println(nachname
        + ", Anzahl Buchstaben: "
        + ((String)nachname).length());
}
```



Vorspann

Konventionen

for each

Generics

Funkt. Interfaces / Lambda-Ausdrücke

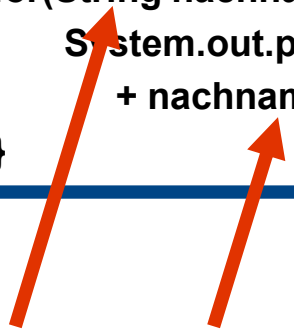
Funktionale Programmierung

Fazit

Generics

Beispiel ArrayList

```
ArrayList<String> nachnamen = new ArrayList<String>();
nachnamen.add("Brabender");
nachnamen.add("Blunck");
nachnamen.add("Oesing");
for(String nachname : nachnamen){
      System.out.println(nachname  + ", Anzahl Buchstaben: "
        + nachname.length());
}
```



Diamand Operator

- Abkürzende Schreibweise seit Java 1.7.

```
ArrayList<String> nachnamen = new ArrayList<>();
```


Vorspann

Konventionen

for each

Generics

Funkt. Interfaces / Lambda-Ausdrücke

Funktionale Programmierung

Fazit

Generics

Beispiel ArrayList

- ArrayList ermöglicht folgendermaßen die Verwendung von Generics.

```
public class ArrayList<E> … {
    …
    public E get(int index){
        …
    }
    …

    public E set(int index, E element){
        …
    }
    …
}
```


Vorspann

Konventionen

for each

Generics

Funkt. Interfaces / Lambda-Ausdrücke

Funktionale Programmierung

Fazit

Generics

Vorspann
Konventionen
for each
Generics
Funkt. Interfaces / Lambda-Ausdrücke
Funktionale Programmierung
Fazit

Klassen mit Generics selbst erstellen

```
public class Wert<E> {

    private E wert;

    public void setzeWert(E wert){
        this.wert = wert;
    }

    public void gibWertAus(){
        System.out.println(
            "Der Wert ist: "
            + this.wert);
    }
}
```

```
public class Anwendungssystem {

    pubic static void main(String[ ] args){
        Wert<String> nachname
            = new Wert<String>();
        nachname.setzeWert("Oesing");
        nachname.gibWertAus();
        Wert<Integer> geburtsjahr
            = new Wert<Integer>();
        geburtsjahr.setzeWert(1970);
        geburtsjahr.gibWertAus();
    }
}
```

```
Der Wert ist: Oesing
Der Wert ist: 1970
```

- Mehrere Generics werden mittels <E1, E2, …> definiert.

Generics

Interfaces mit Generics selbst erstellen

- Analog zu Klassen mit Generics werden Interfaces mit Generics erstellt.

Aufgabe

- Ändern Sie das Beispiel zu Klassen mit Generics ab. Erstellen Sie ein Interface WertInterface, welches von der Klasse Wert implementiert wird. WertInterface enthält die abstrakten Methoden public void setzeWert(E wert) und public void gibWertAus().

  Erstellen Sie dann in der main-Methode Variablen vom Typ des Interfaces.

Vorspann
Konventionen
for each
Generics
Funkt. Interfaces / Lambda-Ausdrücke
Funktionale Programmierung
Fazit

Generics

Methoden mit Generics selbst erstellen

```
public class AnwendungssystemMethode {

    private <T> int findeIndexVonElement(
        List<T> liste, T element){
        int i = 0; int index = -1;
        while(i < liste.size() && index == -1){
            if(liste.get(i).equals(element)){
                index = i;
            }
            i++;
        }
        return index;
    }
    …
```

Generics werden bekannt gemacht.

```
    …

    pubic static void main(String[ ] args){
        List<Integer> geburtsjahre
            = new ArrayList<>();
        geburtsjahre.add(1965);
        geburtsjahre.add(1970);
        geburtsjahre.add(1980);
        int pos = findeIndexVonElement(
            geburtsjahre, 1980);
        System.out.println("Position von "
            + "1980 ist " + pos);
    }
}
```

- Mehrere Generics werden mittels <T1, T2, …> definiert.

Vorspann

Konventionen

for each

Generics

Funkt. Interfaces / Lambda-Ausdrücke

Funktionale Programmierung

Fazit

Generics

Vorspann

Konventionen

for each

Generics

Funkt. Interfaces / Lambda-Ausdrücke

Funktionale Programmierung

Fazit

Invarianz bei Generics

- Die Ableitungsbeziehungen von Elementen überträgt sich nicht auf generische Klassen.
- Integer ist abgeleitet von Number aber nicht ArrayList<Integer> von ArrayList<Number>:
  ArrayList<Number> a = new ArrayList<Integer>(); //Kompilierfehler

Typeinschränkungen mittels extends

- public class Wert<E extends Number> { … }
  Es können nur Objekte mit Elementen vom Typ Number oder Ableitungen von Number definiert werden.
  Wert<String> nachname = new Wert<String>(); //Kompilierfehler
  Wert<Integer> geburtsjahr = new Wert<Integer>(); //OK

vgl. https://www.torsten-horn.de/techdocs/java-generics.htm

Generics

Fortsetzung:Typeinschränkungen mittels extends

- Es gibt kein implements. Die Implementierung eines Interfaces kann aber gefordert werden.
  public class NameGenerischeKlasse<E extends NameInterface> { … }
- Mittels <E extends NameTyp1 & NameTyp2 & … >
  kann die Implementierung mehrerer Interfaces gefordert werden.
  Typ1 darf auch eine Klasse sein.

vgl. https://www.torsten-horn.de/techdocs/java-generics.htm

Vorspann
Konventionen
for each
Generics
Funkt. Interfaces / Lambda-Ausdrücke
Funktionale Programmierung
Fazit

Generics

Aufgabe zur Typeinschränkung

- Gegeben seien die Klassen Person und PersonMitGeburtsjahr.

```
public class Person {

    private String vorname;
    private String nachname;

// Standard-Konstruktor
    public Person(String vorname,
        String nachname){
        super();
        this.vorname = vorname;
        this.nachname = nachname;
    }

    // get- und set-Methoden
    …
}
```

```
public class PersonMitGeburtsjahr
    extends Person{

    private int geburtsjahr;

    // Standard-Konstruktor
    public PersonMitGeburtsjahr(
        String vorname, String nachname,
        int geburtsjahr){
        super(vorname, nachname);
        this.geburtsjahr = geburtsjahr;
    }

    // get- und set-Methode
    …
}
```


Vorspann

Konventionen

for each

Generics

Funkt. Interfaces / Lambda-Ausdrücke

Funktionale Programmierung

Fazit

Generics

Aufgabe zur Typeinschränkung

- Gegeben sei das Interface Berechne2000.

```
public interface Berechne2000 {

    public boolean istKleinerAls();
    public String gibAus();
}
```

- Erweitern Sie die Klasse PersonMitGeburtsjahr.
  Sie soll das Interface Berechne2000 implementieren.
  Es soll geprüft werden, ob das Geburtsjahr kleiner als 2000 ist, und es sollen Vorname, Nachname, Geburtsjahr und das Ergebnis der Prüfung ausgegeben werden.
- Ändern Sie die Klasse Wert, siehe oben, ab. Sie soll ein Generic erhalten, welches von der Klasse Person abgeleitet sein muss bzw. Person selbst ist und das Interface Berechne2000 implementiert. Die Methode gibWertAus soll Der Wert ist: <Vorname> <Nachname> <Geburtsjahr> : <true/false> ausgeben.
- Passen Sie die Klasse Anwendungssystem, siehe oben, an.

Vorspann
Konventionen
for each
Generics
Funkt. Interfaces / Lambda-Ausdrücke
Funktionale Programmierung
Fazit

Generics

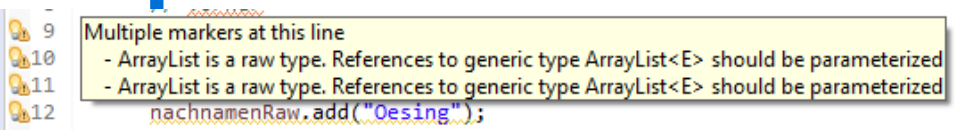
Generics mit wildcards

vgl. https://www.torsten-horn.de/techdocs/java-generics.htm#Wildcards

`<?>`	Beliebiger Typ (vermeidet Compiler-Warnung) ("unbounded wildcard")
`<? super [Type]>`	Superklasse von `[Type]` ("lower bounded wildcard")
`<? extends [Type]>`	Von `[Type]` (Klasse oder Interface) abgeleiteter Typ ("upper bounded wildcard")
`<? extends [Type1] & [IType2] & [IType3]>`	Subtyp von mehreren Typen (außer dem ersten Typ nur Interfaces erlaubt) ("multiple bounds")

- **BeliebigerTyp <?>** ergibt

```
ArrayList nachnamen
    = new ArrayList();
```



- Abhilfe:

```
ArrayList<?> nachnamen = new ArrayList<>();
```

- Vorteil: Compiler wirft Warnungen zu möglichen ClassCastExceptions …, ArrayList<?> steht für eine homogene Liste eines unbekannten Typs.

Vorspann
Konventionen
for each
Generics
Funkt. Interfaces / Lambda-Ausdrücke
Funktionale Programmierung
Fazit

Generics

➢ Upper bounded wildcard <? extends [Type]>

- wie <T extends [Type]>,

```
private <T extends Number> Number
    holeElement(List<T> liste, int position){
    return liste.get(position);
}
```

```
private Number holeElement(
      List<? extends Number> liste,
      int position){
     return liste.get(position);
}
```

nur kann der Typ T bei <? extends [Type]> nicht verwendet werden, da nicht deklariert.

Vorspann
Konventionen
for each
Generics
Funkt. Interfaces / Lambda-Ausdrücke
Funktionale Programmierung
Fazit

Generics

Lower bounded wildcard <? super [Type]>

- Es gebe zwei Klassen Person und PersonMitGeburtsjahr, wobei Person-MitGeburtsjahr von Person abgeleitet ist, Attribute und Konstruktoren wie im Beispiel oben. Mit Hilfe der Methode toString werden die Werte der Attribute als String zurückgegeben.
- Es können nur Objekte mit Elementen vom Typ Person oder vom Typ einer Basisklasse von Person übergeben werden.

```
private Object holePerson(List<? super Person> personen, int position){
    return personen.get(position);
}
```

personen.get(position) könnte vom Typ Object sein!

```
// Kompilierfehler List<PersonMitGeburtsjahr> personen = new ArrayList<>();
List<Object> personen = new ArrayList<>();
 …
System.out.println(holePerson(personen, 0).toString());
```

Vorspann
Konventionen
for each
Generics
Funkt. Interfaces / Lambda-Ausdrücke
Funktionale Programmierung
Fazit

Generics

PECS

- *Producer Extends, Consumer Super*

vgl. http://www.angelikalanger.com/GenericsFAQ/FAQSections/TypeArguments.html#FAQ103

```
public class Collections{

    public static <T> void copy(List<? super T> dest,
        List<? extends T> src){
        for(int i = 0; i < src.size(); i++){
            dest.set(i, src.get(i));
        }
    }
}
```

Vorspann

Konventionen

for each

Generics

Funkt. Interfaces / Lambda-Ausdrücke

Funktionale Programmierung

Fazit

Funktionale Interfaces / Lambda-Ausdrücke

Vorspann

Konventionen

for each

Generics

Funkt. Interf. / Lambda-Ausdrücke

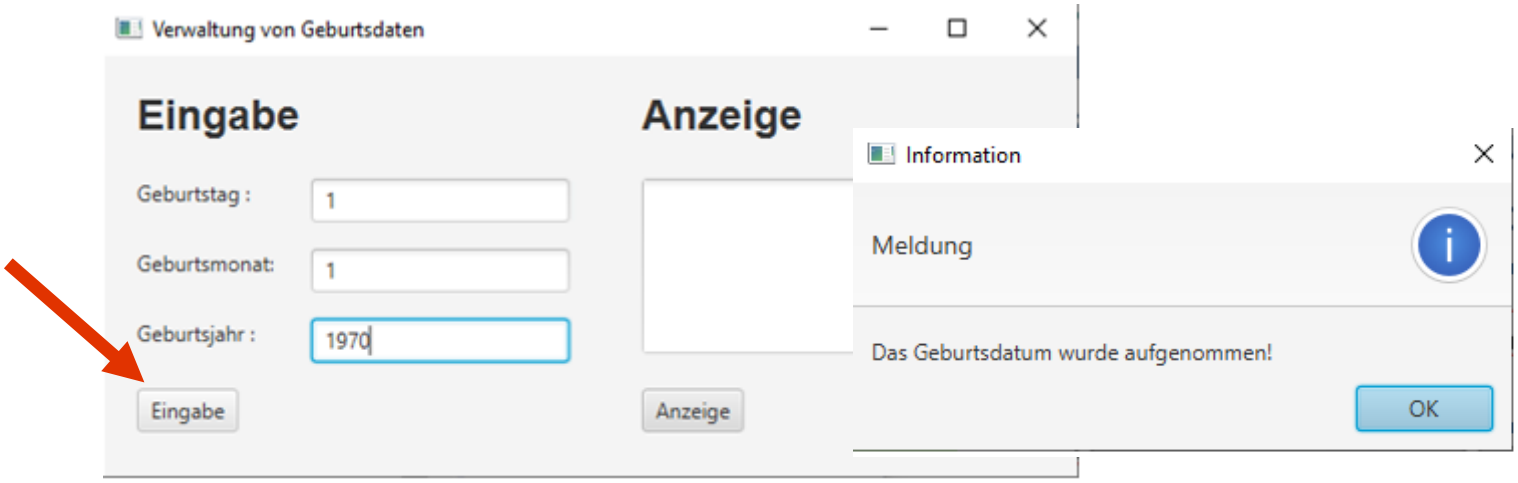
Funktionale Programmierung

Fazit

Funktionale Interfaces

- Interface mit genau einer abstrakten Methode (bis auf default-Methode).
- Beispiel für ein Funktionales Interface:

```
public interface EventHandler<T extends Event> extends EventListener{
    public void handle(T event);
}
```



Funktionale Interfaces / Lambda-Ausdrücke

Innere Klasse

- public final void setOnAction(EventHandler<ActionEvent> value)

Vorspann

Konventionen

for each

Generics

Funkt. Interf. / Lambda-Ausdrücke

Funktionale Programmierung

Fazit


```
public class Anwendungssystem{
    …
    private void initListener(){
        btnEingabe.setOnAction(new EinEventHandler());
        …
    }

    private class EinEventHandler implements EventHandler<ActionEvent>{
        @Override
        public void handle(ActionEvent e){
            nehmeGeburtsdatumAuf();
        }
        …
    }
}
```

Funktionale Interfaces / Lambda-Ausdrücke

Innere anonyme Klasse

- public final void setOnAction(EventHandler<ActionEvent> value)



```
public class Anwendungssystem{
    …
    private void initListener(){
        btnEingabe.setOnAction(new EventHandler<ActionEvent>(){
            @Override
            public void handle(ActionEvent e){
                nehmeGeburtsdatumAuf();
            }
        } );
        …
    }
}
```


Vorspann

Konventionen

for each

Generics

Funkt. Interf. / Lambda-Ausdrücke

Funktionale Programmierung

Fazit

Funktionale Interfaces / Lambda-Ausdrücke

Vorspann
Konventionen
for each
Generics
Funkt. Interf. / Lambda-Ausdrücke
Funktionale Programmierung
Fazit

Lambda-Ausdruck

```
private void initListener(){
    btnEingabe.setOnAction(new EventHandler<ActionEvent>(){
        @Override
        public void handle(ActionEvent e){
            nehmeGeburtsdatumAuf();
        }
    } );
}
```

```
private void initListener(){
    btnEingabe.setOnAction(e -> nehmeGeburtsdatumAuf());
}
```

- Lambda-Ausdruck für die Implementierung eines Funktionalen Interfaces.
- Lambda-Ausdruck zur Vermeidung von inneren anonymen Klassen, keine Objektbildung mit new!

Funktionale Interfaces / Lambda-Ausdrücke

Vorspann
Konventionen
for each
Generics
Funkt. Interf. / Lambda-Ausdrücke
Funktionale Programmierung
Fazit

Lambda-Ausdruck

- Beispiel

```
private void initListener(){
    btnEingabe.setOnAction(e -> nehmeGeburtsdatumAuf());
}
```

Methodenargument

Anweisung des Methodenrumpfes

Allgemeine Syntax

```
Objekt obj = (e1, e2, …, eq) -> { Anweisung1;
                                  Anweisung2;
                                  …
                                  Anweisungr;};
```

- Gibt es keine Methodenargumente, so ist die Syntax **() -> {…}** .

Funktionale Interfaces / Lambda-Ausdrücke

Vorspann

Konventionen

for each

Generics

Funkt. Interf. / Lambda-Ausdrücke

Funktionale Programmierung

Fazit

Aufgabe

- Erstellen Sie ein Funktionales Interface Hintereinander-haengen mit einer Methode haengeHintereinander, die als Eingabeparameter zwei Strings besitzt und als Rückgabewert einen String.
- Erstellen Sie eine Anwendung mit einer main-Methode, in welcher ein Objekt erstellt wird, welches das Funktionale Interface mit Hilfe eines Lambda-Ausdrucks implementiert. Die beiden Strings sollen hintereinander gehängt werden. Das Ergebnis soll in einer zweiten Anweisung als Rückgabewert zurückgegeben werden.
- Mit Hilfe dieses Objekts sollen zwei Strings Elke und Musterfrau in die Konsole geschrieben werden:

  Elke Musterfrau.

Funktionale Interfaces / Lambda-Ausdrücke

Vorspann

Konventionen

for each

Generics

Funkt. Interf. / Lambda-Ausdrücke

Funktionale Programmierung

Fazit

Bemerkung

- Funktionen können also in Form von Lambda-Ausdrücken wie Werte behandelt werden. Variablen können als Wert Funktionen enthalten.

Fortsetzung der Aufgabe

- Erweitern Sie die Anwendung um eine finale Variable anrede, die Sie mit Frau belegen. Diese soll für die Erweiterung der Ausgabe innerhalb des Lambda-Ausdrucks verwendet werden: Frau Elke Musterfrau.

Funktionale Interfaces / Lambda-Ausdrücke

Vorspann

Konventionen

for each

Generics

Funkt. Interf. / Lambda-Ausdrücke

Funktionale Programmierung

Fazit

Ergänzungen zu Funktionalen Interfaces

Supplier

- Anbieter, Lieferant
- Funktionales Interface, welches genau eine Methode, nämlich T get() enthält.

```
package java.util.function;

@FunctionalInterface
public interface Supplier<T>{
    T get();
}
```

BooleanSupplier

- Interface, welches genau eine Methode, nämlich boolean getAsBoolean() enthält.
- Analog zum BooleanSupplier gibt es einen DoubleSupplier, IntSupplier und LongSupplier.

```
package java.util.function;

@FunctionalInterface
public interface BooleanSupplier{
    boolean getAsBoolean();
}
```

Funktionale Interfaces / Lambda-Ausdrücke

Ergänzungen zu Funktionalen Interfaces: Beispiel zu Supplier<String>

```
public class NameSupplier
    implements Supplier<String>{

    private Person person;

    public NameSupplier(
        Person person) {
        this.person = person;
    }
    @Override
    public String get() {
        return this.person.getVorname()
            + "  "
            + this.person.getNachname();
    }
}
```

```
public class Anwendungssystem{

    public static void main(String[ ] args){
        NameSupplier nameSupplier
            = new NameSupplier(
            new Person("Elke" , "Musterfrau"));
        System.out.println(nameSuppler.get());
    }
}
```



- Supplier macht Sinn, wenn es der Methode, die den Supplier aufruft, egal ist, wie die Daten erzeugt werden.

Funktionale Interfaces / Lambda-Ausdrücke

Vorspann

Konventionen

for each

Generics

Funkt. Interf. / Lambda-Ausdrücke

Funktionale Programmierung

Fazit

Aufgabe

- Gegeben sei eine Klasse Person mit den Attributen vorname und nachname und üblichen Konstruktor, get- und set-Methoden.
  Gegeben sei weiterhin eine Klasse PersonMitGeburtsjahr, die von Person erbt und ein weiteres Attribut geburtsjahr hat mit üblichem Konstruktor, get- und set-Methode. Gegeben sei die folgende Klasse

```
public class Anwendungssystem{

    public static void main(String[ ] args){
        PersonMitGeburtsjahr p
            = new PersonMitGeburtsjahr("Elke" , "Musterfrau",1970);
        if(p.getGeburtsjahr() < 2000){
            System.out.println(("Geburtsjahr ist kleiner als 2000.");
        }
        else{
            System.out.println(("Geburtsjahr ist größer oder gleich 2000.");
        }
    }
}
```

Funktionale Interfaces / Lambda-Ausdrücke

Vorspann

Konventionen

for each

Generics

Funkt. Interf. / Lambda-Ausdrücke

Funktionale Programmierung

Fazit

Aufgabe

- Erstellen Sie eine Klasse PersonSupplier, welche einen BooleanSupplier implementiert. Sie soll ein Attribut vom Typ PersonMitGeburtsjahr besitzen, welches in einem Konstruktor initialisiert wird. Weiterhin soll true herausgegeben werden, falls das Geburtsjahr der Person kleiner als 2000 ist, false sonst.
- Ändern Sie die vorgegebene Klasse Anwendungssystem ab. Sie soll die bisherige Funktionalität mit Hilfe der Klasse PersonSupplier ausführen.

Funktionale Interfaces / Lambda-Ausdrücke

Vorspann

Konventionen

for each

Generics

Funkt. Interf. / Lambda-Ausdrücke

Funktionale Programmierung

Fazit

Aufgabe

- Ändern Sie die vorgegebene Klasse Anwendungssystem ab. Sie soll die bisherige Funktionalität weiterhin ausführen, aber den BooleanSupplier mit Hilfe eines Lambda-Ausdrucks implementieren. (Die Klasse PersonSupplier wird dann nicht mehr benötigt.)
- Lagern Sie dazu den if-else-Abschnitt mit der Konsolen-ausgabe in einer privaten statischen Methode void erstelleAusgabe(BooleanSupplier booleanSupplier) aus.

Notiz

- Der Methode erstelleAusgabe ist egal, wie das Ergebnis von getAsBoolean von booleanSupplier zustande kommt.

Funktionale Programmierung

Vorspann

Konventionen

for each

Generics

Funkt. Interfaces / Lambda-Ausdrücke

Funktionale Programmie-rung

Fazit

Programmiertechnik (u.a.)	Grundlage für Prinzipien

Funktionale Programmierung

Vorspann

Konventionen

for each

Generics

Funkt. Interfaces / Lambda-Ausdrücke

Funktionale Programmie-rung

Fazit

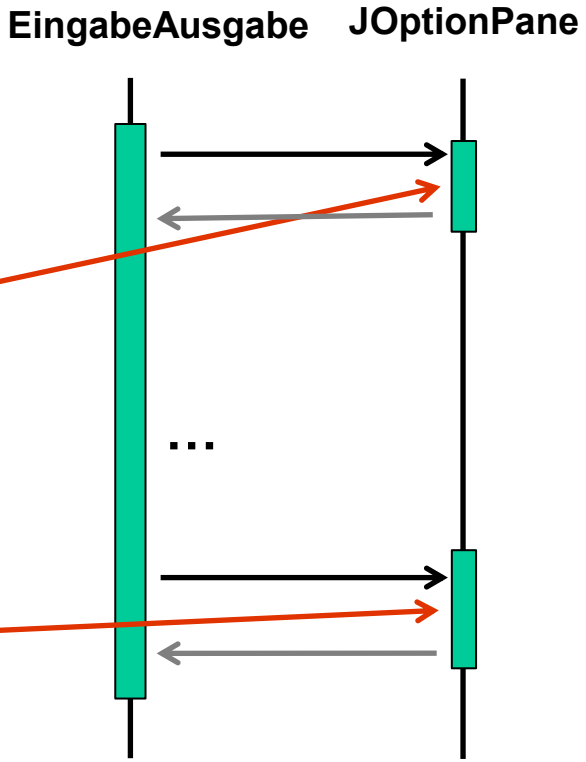
Wieso Vertiefung Java?

Programmiertechnik (u. a.)	Grundlage für Prinzipien
Auslagerung von Quellcode mit Hilfe von Funktionen / Methoden auch in andere Dateien	Modulare Programmierung

```
import javax.swing.*;

public class EingabeAusgabe {


   public static void main(String[ ] args){
        String tag = JOptionPane
             .showInputDialog("Geburtstag");
        ...
        String ergebnis
            = tag + ". " + monat + ". " + jahr;
        JOptionPane
            .showMessageDialog(null, ergebnis);
   }
}
```



Funktionale Programmierung

Wieso Vertiefung Java?

Programmiertechnik (u. a.)	Grundlage für Prinzipien
Auslagerung von Quellcode mit Hilfe von Funktionen / Methoden auch in andere Dateien	Modulare Programmierung

Vorspann

Konventionen

for each

Generics

Funkt. Interfaces / Lambda-Ausdrücke

Funktionale Programmie-rung

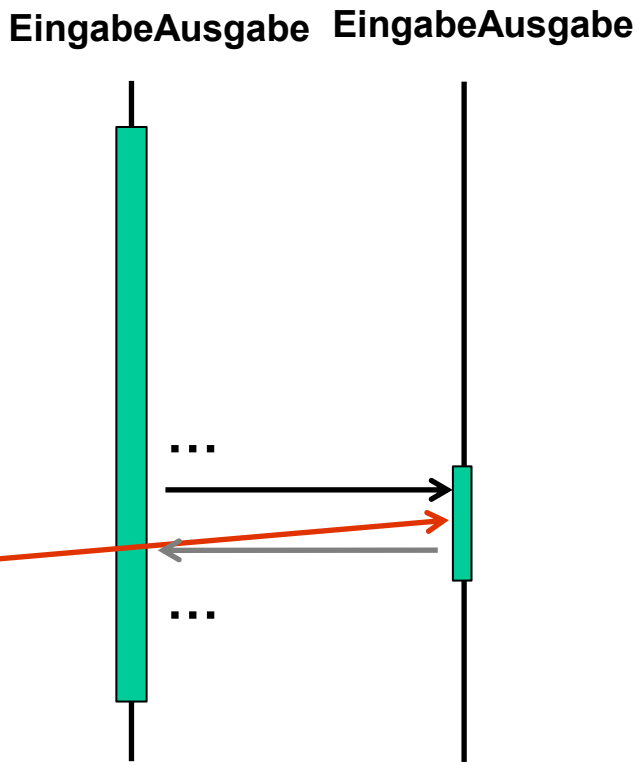
Fazit


```
import javax.swing.*;

public class EingabeAusgabe {

   public static String gibDatumZurueck(
        String tag, String monat, String jahr){
        return tag + ". " + monat + ". " + jahr;
   }

   public static void main(String[ ] args){
        …
        String ergebnis
             = gibDatumZurueck(tag, monat, jahr);
        …
   }
}
```



Funktionale Programmierung

Wieso Vertiefung Java?

Programmiertechnik (u. a.)	Grundlage für Prinzipien
Eigene Datentypen (structs in C, Records in Pascal)	

```
...
public class EingabeAusgabe {
   ...
   public static String gibDatumZurueck(Geburtsdatum gebDatum){
        return gebDatum.geburtstag + ". " + gebDatum.geburtsmonat
             + ". " + gebDatum.geburtsjahr;
  }
  ...
  public static void main(String[ ] args){
       String tag = JOptionPane.showInputDialog("Geburtstag");
        ...
        Geburtsdatum gebDatum = new Geburtsdatum();
        gebDatum.geburtstag = Integer.parseInt(tag);
        gebDatum.geburtsmonat = Integer.parseInt(monat);
        gebDatum.geburtsjahr = Integer.parseInt(jahr);
        String ergebnis = gibDatumZurueck(gebDatum);
        ...
  } }
```

```
public class Geburtsdatum {
     static int geburtstag;
     static int geburtsmonat;
     static int geburtsjahr;
}
```

Funktionale Programmierung

Wieso Vertiefung Java?

Programmiertechnik (u. a.)	Grundlage für Prinzipien
Klassen als Bauplan für Objekte mit Methoden	Objektorientierte Programmierung

```
public class Geburtsdatum {
    static int geburtstag;
    static int geburtsmonat;
    static int geburtsjahr;
}
```

```
public class Geburtsdatum {

    private int geburtstag;
    private int geburtsmonat;
    private int geburtsjahr;

    // Konstruktor, welcher Plausibilitaeten ueberprueft
    // und die Attribute mit Werten belegt
    …
    // get- und set-Methoden, Methoden zur
    // Ueberpruefung von Plausibilitaeten
    …
    public String gibGeburtsdatumZurueck(){
        return this.getGeburtstag() + ". "
            + this.getGeburtsmonat () + ". "
            + ". " + this.getGeburtsjahr();
    }
}
```


Vorspann

Konventionen

for each

Generics

Funkt. Interfaces / Lambda-Ausdrücke

Funktionale Programmie-rung

Fazit

Funktionale Programmierung

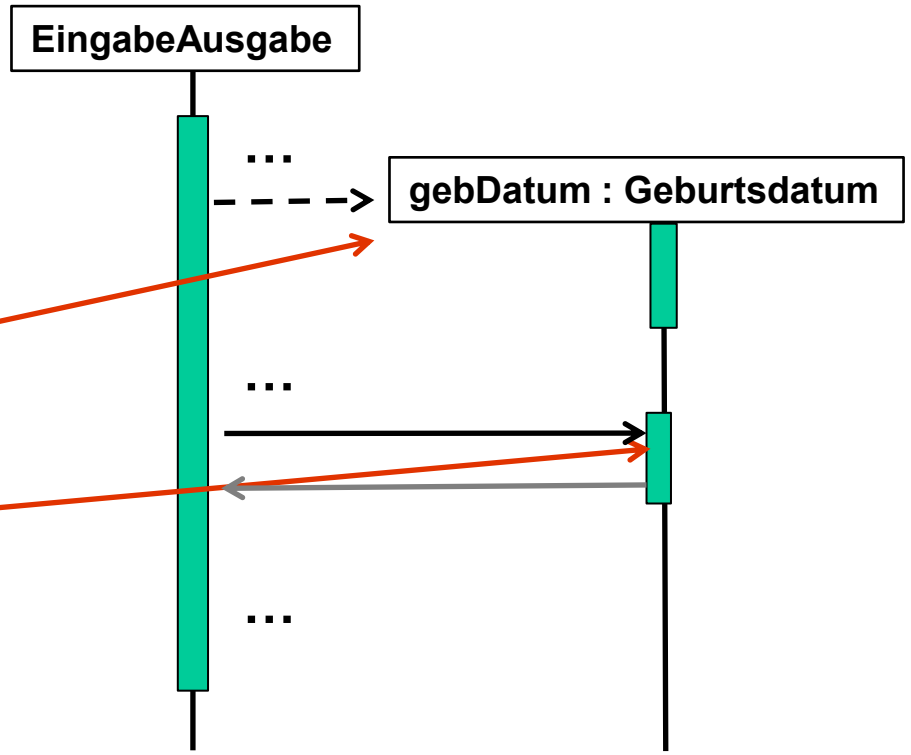
Wieso Vertiefung Java?

Programmiertechnik (u. a.)	Grundlage für Prinzipien
Klassen als Bauplan für Objekte mit Methoden	Objektorientierte Programmierung

```
import javax.swing.*;

public class EingabeAusgabe {

   public static void main(String[ ] args){
        String tag = JOptionPane
             .showInputDialog("Geburtstag");
        …
        Geburtsdatum gebDatum
             = new Geburtsdatum(tag, monat, jahr);
        String ergebnis
             = gebDatum.gibGeburtsdatumZurueck();
        JOptionPane
            .showMessageDialog(null, ergebnis);
   }
}
```



Funktionale Programmierung

Vorspann

Konventionen

for each

Generics

Funkt. Interfaces / Lambda-Ausdrücke

Funktionale Programmie-rung

Fazit

Funktionale Paradigma

- **Imperatives Paradigma**
  Ein Programm wird als Berechnung eines Sachverhalts aufgefasst, die mittels einer Folge von Anweisungen durchgeführt wird. Wie gelangt man zu dem Ergebnis?
- **Funktionale Programmierung**
  Funktionen / Methoden werden nicht nur definiert und angewendet, sondern auch wie Daten miteinander verknüpft und können auch als Parameter verwendet werden. Ausdrücke sind die Erzeugung von Funktionen (Lambda-Ausdrücke) und die Anwendung von Funktionen.
- **Bemerkung**
  Java erlaubt das Programmieren nach unterschiedlichen Paradigmen, imperativ, deklarativ und objektorientiert.

Funktionale Programmierung

Vorspann

Konventionen

for each

Generics

Funkt. Interfaces / Lambda-Ausdrücke

Funktionale Programmie-rung

Fazit

Funktionale Konzepte

Es gibt eine Reihe von Konzepten, die typisch für funktionale Sprachen sind.

- **Funktionen höherer Ordnung**
  Funktion, die ihrerseits Funktionen als Argument erwartet, oder eine Funktion zurück gibt.

```
private void initListener(){
    EventHandler<ActionEvent> eHandler = e -> nehmeGeburtsdatumAuf();
    btnEingabe.setOnAction(eHandler);
}
```

- Funktionen können im Programm überall dort auftauchen, wo auch primitive Werte wie Zahlen oder Strings auftauchen können.

Funktionale Programmierung

Funktionale Konzepte

- Funktionen können im Programm überall dort auftauchen, wo auch primitive Werte wie Zahlen oder Strings auftauchen können.

```
public class Anwendungssystem{

    public static void main(String[ ] args){
        String anrede = "Frau";
        Hintereinanderhaengen h
            = (string1, string2) -> {
                String ergebnis = anrede + " "
                    + string1 + " "  + string2;
                return ergebnis;
            };
        System.out.println(h.haengeHintereinander(
            "Elke", "Musterfrau"));
    }
}
```



```
public interface Hintereinanderhaengen {

    public String haengeHintereinander(
            String string1, String string2);
}
```

Funktionale Programmierung

Funktionale Konzepte

- Funktionen können im Programm überall dort auftauchen, wo auch primitive Werte wie Zahlen oder Strings auftauchen können.

```
public class Anwendung{

    public static void main(String[ ] args){
        Ball ball = new Ball(true, true);
        // BooleanSupplier bsBall = () -> { ball.isSpielball() };
        BooleanSupplier bsBall = ball :: isSpielball;
        if(bsBall.getAsBoolean()){
            System.out.println("Der Ball ist ein Spielball!");
        } else {
            System.out.println("Der Ball ist ein Trainingsball!");
        }
    }
}
```



Methodenreferenz

```
public class Ball{

    private boolean intakt;
    private boolean spielball;
    …
    public Ball(boolean intakt,
        boolean spielball){
        this.intakt = intakt;
        this.spielball = spielball;
    }

    public boolean isIntakt(){
        return this.intakt;
    }

    public boolean isSpielball(){
        return this.spielball;
    }
    …
}
```

Funktionale Programmierung

Vorspann

Konventionen

for each

Generics

Funkt. Interfaces / Lambda-Ausdrücke

Funktionale Programmie-rung

Fazit

Funktionale Konzepte

- **Pure functions**

  - Der Rückgabewert hängt nur von den Eingabeparametern ab.

    Gegenbeispiel

  - Die Ausführung der Funktion hat keine Seiteneffekte.

    Gegenbeispiel

```
public class Ball{

    private boolean intakt;
    private boolean spielball;
    …
    public Ball(boolean intakt,
        boolean spielball){
        this.intakt = intakt;
        this.spielball = spielball;
    }
    …
    public boolean isSpielball(){
        return this.spielball;
    }

    public void setSpielball(
        boolean spielball){
        this.spielball = spielball;
    }
    …
}
```

Funktionale Programmierung

Funktionale Konzepte

- **Pure functions**
  - Der Rückgabewert hängt nur von den Eingabeparametern ab.
  - Die Ausführung der Funktion hat keine Seiteneffekte.

```
public class Ball{

    private boolean intakt;
    private boolean spielball;
    …
    public boolean isIntakt(){
        return this.intakt;
    }
    …
    public boolean isSpielball(){
        return this.spielball;
    }

    public boolean istIntakterSpielball(Ball ball){
        return ball.isIntakt() && ball.isSpielball();
    }
}
```


Vorspann

Konventionen

for each

Generics

Funkt. Interfaces / Lambda-Ausdrücke

Funktionale Programmie-rung

Fazit

Fazit

Wieso Vertiefung Java?

- **Konventionen** werden in der Zusammenarbeit im Entwicklungsteam benötigt!

Vorspann

Konventionen

for each

Generics

Funkt. Interfaces / Lambda-Ausdrücke

Funktionale Programmierung

Fazit

Fazit

Wieso Vertiefung Java?

- Weiterentwicklungen in Java haben sich in der Praxis als notwendig erwiesen:
  - umständliche Programmierung von Fachlichkeit, da bisher keine geeigneten Datentypen vorhanden (**Enumeration**, **Generics**)

Vorspann

Konventionen

for each

Generics

Funkt. Interfaces / Lambda-Ausdrücke

Funktionale Programmierung

Fazit

Fazit

Wieso Vertiefung Java?

- Weiterentwicklungen in Java haben sich in der Praxis als notwendig erwiesen:
  - umständliche Programmierung, da bisher keine abkürzenden Techniken vorhanden (**for each**, **Lambda-Ausdrücke**)

Vorspann
Konventionen
for each
Generics
Funkt. Interfaces / Lambda-Ausdrücke
Funktionale Programmierung
Fazit

Fazit

Wieso Vertiefung Java?

- Mit der **Funktionalen Programmierung** ist ein ein neuer fundamentaler Programmierstil eingeführt worden.
  Ihnen soll bekannt sein, dass es diese Weiterentwicklungen gibt!

ObjektorientierteProgrammiertechniken_3_Jax.docx zeigen

Vorspann
Konventionen
for each
Generics
Funkt. Interfaces / Lambda-Ausdrücke
Funktionale Programmierung
Fazit

Vertiefung Java

- Vorspann ✓
- Konventionen ✓
- for each ✓
- Generics ✓
- Funktionale Interfaces / Lambda-Ausdrücke ✓
- Funktionale Programmierung ✓
- Fazit ✓

Hochschule Bochum
TECHNIK WIRTSCHAFT GESUNDHEIT

Objektorientierte Programmiertechniken

```
<?>
<? super [Type]>
<? extends [Type]>
<? extends [Type1] & [IType2] & [IType3]>
```

https://www.torsten-horn.de/techdocs/java-generics.htm#Wildcards

Vielen Dank für Ihr Interesse!